PROGRAM ANALYSIS, BIG-O & ADT

AGENDA

Introduction to program analysis

Growth of functions

Notions of algorithm complexity

Benchmarking and efficiency

Data structures, Abstract Data Types (ADT), and Pre/Post Conditions

MENTIMETER FOR TODAY

Instructions

Go to

www.menti.com

Enter the code

3339 3073

Or use QR code

REFLECTION

Consider the following problem:

$$a + b + c = n$$

Find all sets of nonnegative integers (a, b, c) that sum to integer n $(n \geq 0)$

And the following two solutions, made by Alice and Bob (the pseudo code is presented, but both Bob and Alice use only pen and paper to solve the problem for a given n):

Bob's Solution

1. Try all combinations of (a, b, c)
2. If $a + b + c = n$, print(a, b, c)

Alice's Solution

1. For all (a, b) set $c = n - (a + b)$
2. If $c \geq 0$, print(a, b, c)

Discuss:

1. Which one is the most efficient of the two, and why?
2. Did you consider the fact that Bob (respectively Alice) might be faster with pen and paper than Alice (resp. Bob)? If so, why? If not, why?

INTRODUCTION TO PROGRAM ANALYSIS

We Separate *analysis* from *implementation* :

- What is the **abstract** CPU operation executed by the algorithm? (*time)*
- What is the **unit of memory** required to run the algorithm? (*space*)

We define a *model of computation:*

- Simple operations take **1 unit of time**
- Simple variables take **1 unit of storage**

We approximate these for **ideal** input (*best case*), **random** input (*average case*), **bad** input (*worst case*).

EXAMPLE OF ANALYSIS

```
#include <iostream>
#include <vector>
#include <chrono>

#define N 1000

int main() {
int i, j, k;
int result = 0, count = 0;
std::vector<int> a(N), b(N), c(N), d(N);

for (i = 0; i < N; ++i) {
  a[i] = rand() % 100;
  b[i] = rand() % 100;
  ++count; }

for (i = 0; i < N; ++i) {
  for (j = 0; j < N; ++j) {
    for (k = 0; k < N; ++k) {
      a[k] = a[k] * b[k];
      ++count;
    }
    a[j] = a[j] * b[j];
  }
  result += a[i];
}}
```

MENTIMETER - PROGRAM ANALYSIS

A[M]	1	4	7	4	6	12	56	22	4	64	5

B[N]	12	76	13

How many items from B can I find in A?

Method 1

```
For each i in B {
  For each j in A {
    if B[i] == A[j] {
      ++cnt;
      exit();
    }
  }
}
```

Method 2

```
[sort A] // M*log2(M)

For each i in B {
  find B[i] in A //log2(M)
}
```

Method 3

```
[sort A] // M*log2(M)
[sort B] // N*log2(N)
s=0
For each i in B[0..N-1] {
  // log2(s..M-1)
  itr = find B[i] in A[s..M-1]
    if found
      s = itr
}
```

MENTIMETER - BIG-O ANALYSIS!

How many items from B[N] can I find in A[M]?

Method 1

```
For each i in B {
  For each j in A {
    if B[i] == A[j] {
      ++cnt;
      exit();
    }
  }
}
```

```
For each N {
  For each M {
    C
  }
}
```

Method 2

```
[sort A]

For each i in B {
  find B[i] in A[0..M-1]
}
```

```
M*log(M) itr (Quicksort)

For each N {
 C*log(M) (Div-Conquer)
}
```

Method 3

```
[sort A]
[sort B]
s=0
For each i in B[0..N-1] {
  it=find B[i] in A[s..M-1]
    if found
      s = it
}
```

```
N*log(N)
M*log(M)
For each N {
  C*log(M)
  C }
```

INTRODUCTION TO PROGRAM ANALYSIS

A[M]	1	4	7	4	6	12	56	22	4	64	5
B[N]	12	76	13								

How many items from B can I find in A?

Most of algorithms have a **primary parameter** that affects the runtime substantially.

*What is this **primary parameter?***

REFLECTION

What is the primary parameter (N)
in Bob's solution to the problem on the right?

$a + b + c = n$

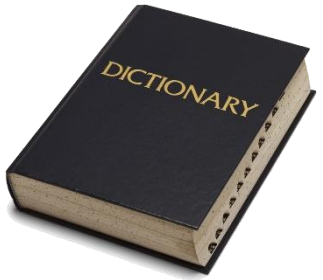
Find all sets of nonnegative integers (a, b, c) that sum to integer n $(n \geq 0)$

Bob's Solution

1. Try all combinations of (a, b, c)
2. If $a + b + c = n$, print(a, b, c)

Imagine that you need to find a word in a dictionary like the one on the right.

- What does your algorithm look like?
- What is the primary parameter (N)?



https://www.collinsdictionary.com/dictionary/english/dictionary

INTRODUCTION TO PROGRAM ANALYSIS

*What is the **best** algorithm to solve a certain problem?*

Analysis is key for understanding algorithms well to apply them to solve practical problems.

However, we cannot do extensive experimentation and deep mathematical analysis of all programs we run.

We need a **basic framework** for performance analysis to **analyze and compare** algorithms so we can apply them to solve our problems.

How can we solve this?

INTRODUCTION TO PROGRAM ANALYSIS

Let us first try **empirical analysis** (implement the algorithms and measure!):

- It takes time to implement all relevant algorithms
- What is the **right** input data? *(real, good, random, ideal, perverse)*
- What is the **right** platform? *(compiler, CPU clock, memory, operating system)*
- What if the problem is **hard** and all algorithms take too much time to run?

There are many **pitfalls** in selecting algorithms:

- **Ignoring** performance characteristics. Faster algorithms are often **more complicated**.
- Paying **too much** attention to performance. *"Premature optimization is the root of all evil." (Don Knuth)*

Resources are finite (your time), spend wisely on improving the algorithms that **matter**.

INTRODUCTION TO PROGRAM ANALYSIS

Just to illustrate the effect of **the compiler**, compile and run the program below with different **optimization settings (-O)** using the time command:

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main(void)
{
  int N;
  long count = 0;
  printf("input N: ");
  scanf("%d", &N);
  int msec = 0, trigger = 10; /* 10ms */
  clock_t before = clock();
  for (int i = 0; i < N; i++) {
    for (int j = 0; j < N; j++) {
      for (int k = 0; k < N; k++) {
        count++;
      }
    }
  }
  clock_t taken = clock()-before;
  msec = taken * 1000/CLOCKS_PER_SEC;
  printf("count = %ld, msek taken = %d", count, msec);
}
```

count.c

```
~/countc$ clang-7 -pthread -o main main.c
~/countc$ ./main
input N: 1000
count = 1000000000, msek taken = 2260
~/countc$ clang-7 -pthread -O1 -o main main.c
~/countc$ ./main
input N: 1000
count = 1000000000, msek taken = 0
~/countc$
```

Without optimization

With optimization

INTRODUCTION TO PROGRAM ANALYSIS

There are many reasons to analyze algorithms:

- To **compare** different algorithms for the same task
- To **predict** performance in a new environment
- To **adjust parameters** for a certain algorithm
- To **know how much resources** are needed to solve the task

Performance characteristics of many algorithms is known, but that is not **always** the case. Discovering that in many cases leads to **new mathematical insights.**

→ **Analysis of Algorithms** is the CS field that studies **complexity analysis** (in *time* and *space*):

- The *analyst* discovers **as much as** possible about an algorithm
- The *programmer* **applies** such information to **select** an algorithm

INTRODUCTION TO PROGRAM ANALYSIS

We now can **separate** *analysis* from *implementation* :

- What is the **abstract** operation executed by the algorithm? (*time)*
- What is the **unit of memory** required to run the algorithm? (*space*)

Hence we can analyze algorithms no matter **how much the abstract operation runs** in nanoseconds or **how many bits an integer has** in a certain machine.

We are implicitly defining a *model of computation:*

- Simple operations take **1 unit of time**
- Simple variables take **1 unit of storage**

We can then **approximately count** these quantities for **ideal** input (*best case*), **random** input (*average case*), **bad** input (*worst case*).

AGENDA

Introduction to program analysis

Growth of functions

Notions of algorithm complexity

Benchmarking and efficiency

Data structures and Abstract Data Types (ADT)

GROWTH OF FUNCTIONS

Most of algorithms have a **primary parameter** N that affects the runtime substantially.

*What is this **primary parameter?***

Depending on the *problem*, the **input size** can be the *degree of a polynomial, the size of a file, the length of a string, the dimensions in an image,* etc.

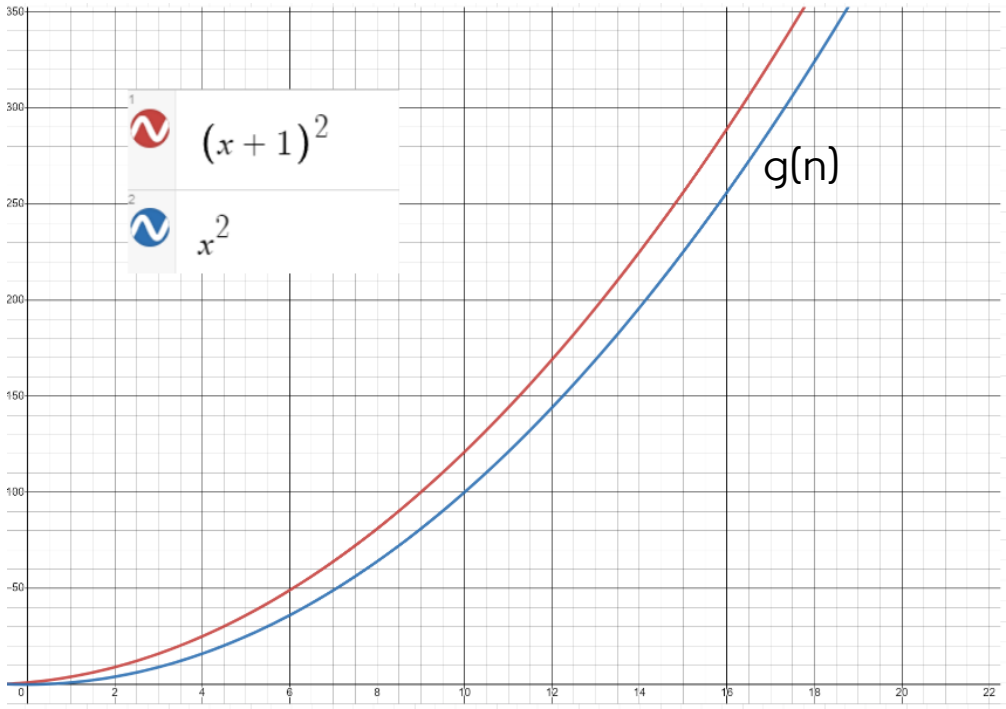
If there are **multiple parameters** (like the dimensions on an image), we express one parameter as a function of the other, so we have a single parameter N.

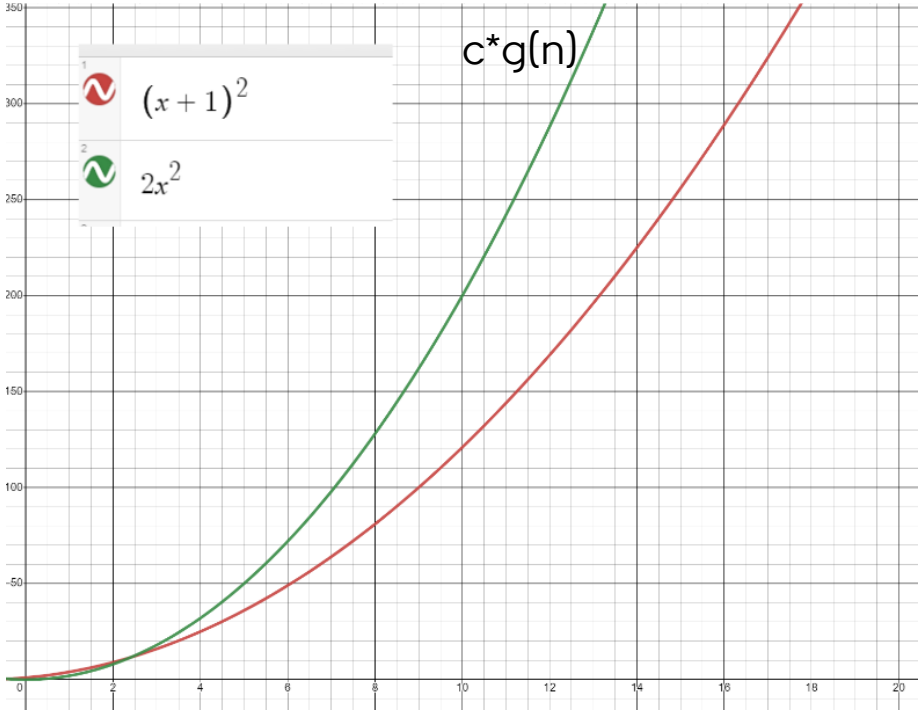
➜ **Objective:** is to express **all the resources** as a function T of just N, typically in the **worst case**.

NOTIONS OF ALGORITHM COMPLEXITY

Example: Consider an algorithm that takes $(n+1)^2$ steps.

Why can we just claim that it is $O(n^2)$, and why not $O((n+1)^2)$? In other words, why do we care about simpler approximations and large n?

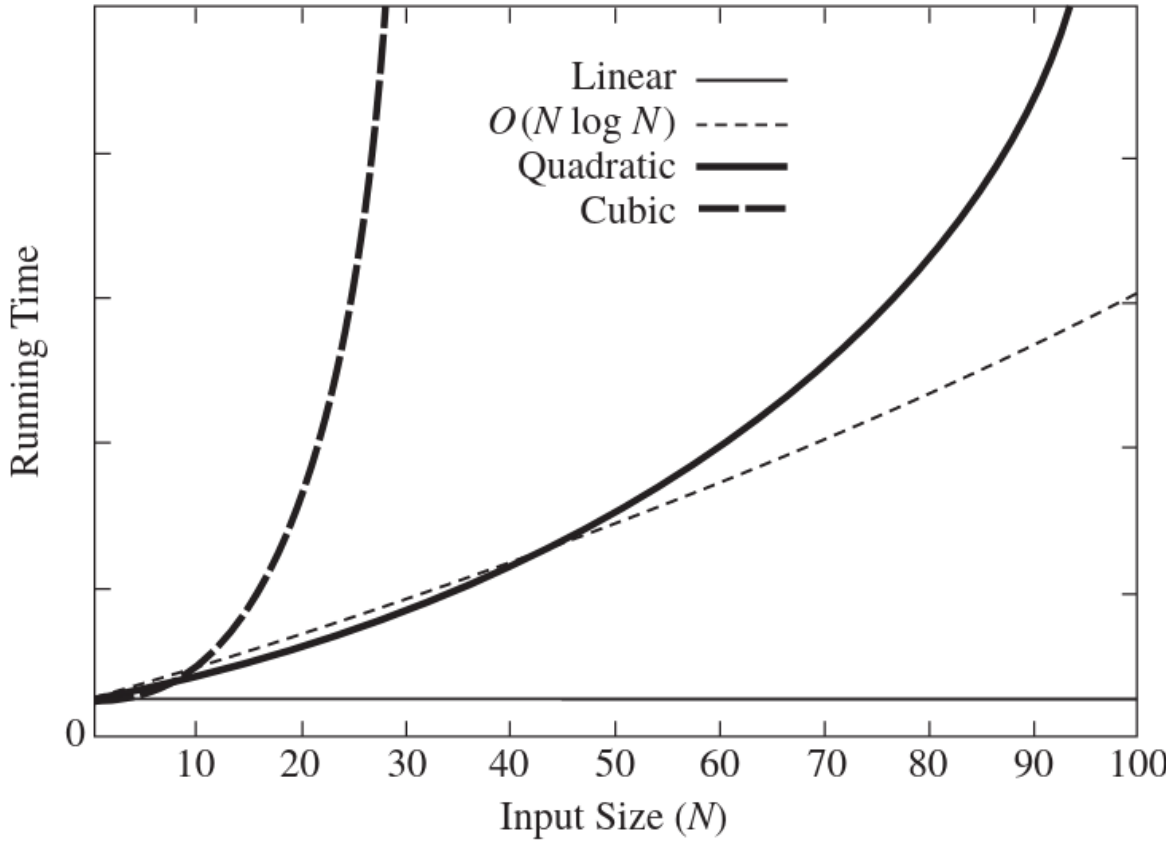


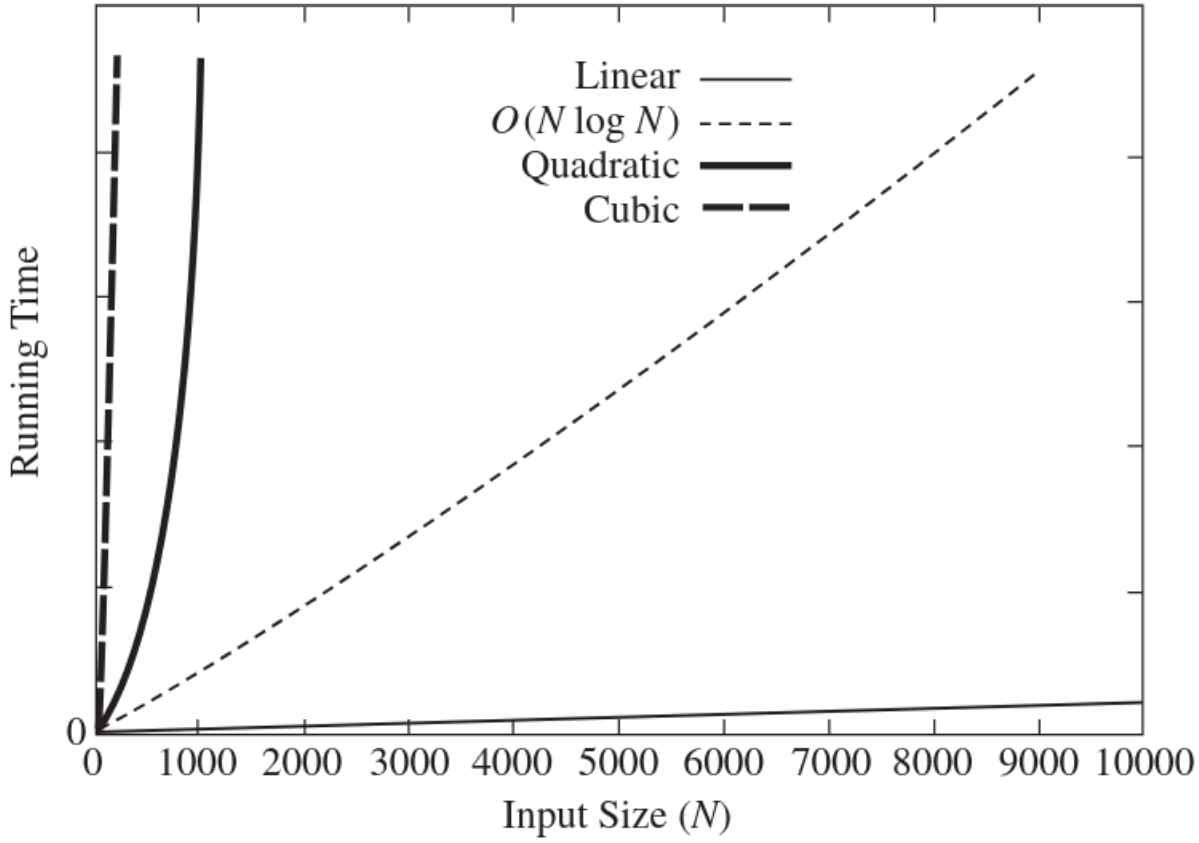


GROWTH OF FUNCTIONS

n	$\log n$	$\sqrt{n}$	n	$n \log n$	n^2	n^3	2^n
2	1	1.4	2	2	4	8	4
4	2	2	4	8	16	64	16
8	3	2.8	8	24	64	512	256
16	4	4	16	64	256	4096	65536
32	5	5.7	32	160	1024	32768	4294967296
64	6	8	64	384	4096	262144	1.84×10^{19}
128	7	11.3	128	896	16384	2097152	3.40×10^{38}
256	8	16	256	2048	65536	16777216	1.16×10^{77}
512	9	22.6	512	4608	262144	134217728	1.34×10^{154}
1024	10	32	1024	10240	1048576	1073741824	

GROWTH OF FUNCTIONS





M=1000, N=1000
B*A = 1000*1000 = 1E6

1000*log2(1000)+
1000*log2(1000) = 20000

1000*log2(1000)+1000*log2(1000)+
log2(1000)+log2(500)+
log2(500)= 19959

GROWTH OF FUNCTIONS

If we can write the number of steps that algorithms take as **functions**, then we can see how they grow as the primary parameter increases!

Note: this restricts the domain of the functions to the **natural numbers!**

For example, see the values *T(N)* for **different** *(N, T):*

$\lg N$	$\sqrt{N}$	N	$N \lg N$	$N(\lg N)^2$	$N^{3/2}$	N^2
3	3	10	33	110	32	100
7	10	100	664	4414	1000	10000
10	32	1000	9966	99317	31623	1000000
13	100	10000	132877	1765633	1000000	100000000
17	316	100000	1660964	27588016	31622777	10000000000
20	1000	1000000	19931569	397267426	1000000000	1000000000000

GROWTH OF FUNCTIONS

What if these are **real programs** running in **real machines**?

operations per second	problem size 1 million			problem size 1 billion		
	N	$N \lg N$	N^2	N	$N \lg N$	N^2
10^6	seconds	seconds	weeks	hours	hours	never
10^9	instant	instant	hours	seconds	seconds	decades
10^{12}	instant	instant	seconds	instant	instant	weeks

➜ **Important:** it is likely that the runtime of a program will involve constants and minor terms. We **ignore** those and consider only the *dominant term.*

GROWTH OF FUNCTIONS

Time will **almost always** proportional to *one of these functions:*

- *Constant (1):* basic operations are executed at most a few times.
- *Logarithmic* or *log(N):* solves problem by breaking in pieces and solving a smaller piece. Notice that base of the log **does not matter.**
- *Linear (N):* a small amount of work is done per input element.
- *Linearithmic* or *Nlog(N):* usually arises in **recursive algorithms**
- *Quadratic (N^2):* solution involves **all pairs** of input elements.
- *Cubic (N^3):* solution involves **all triples** of input elements.
- *Exponential (2^N):* typically brute-force solution by testing **all solutions**.

Important: same idea applies to *space!* In many times, **time and space are closely related.**

GROWTH OF FUNCTIONS

There will be other **functions and constants** as well.

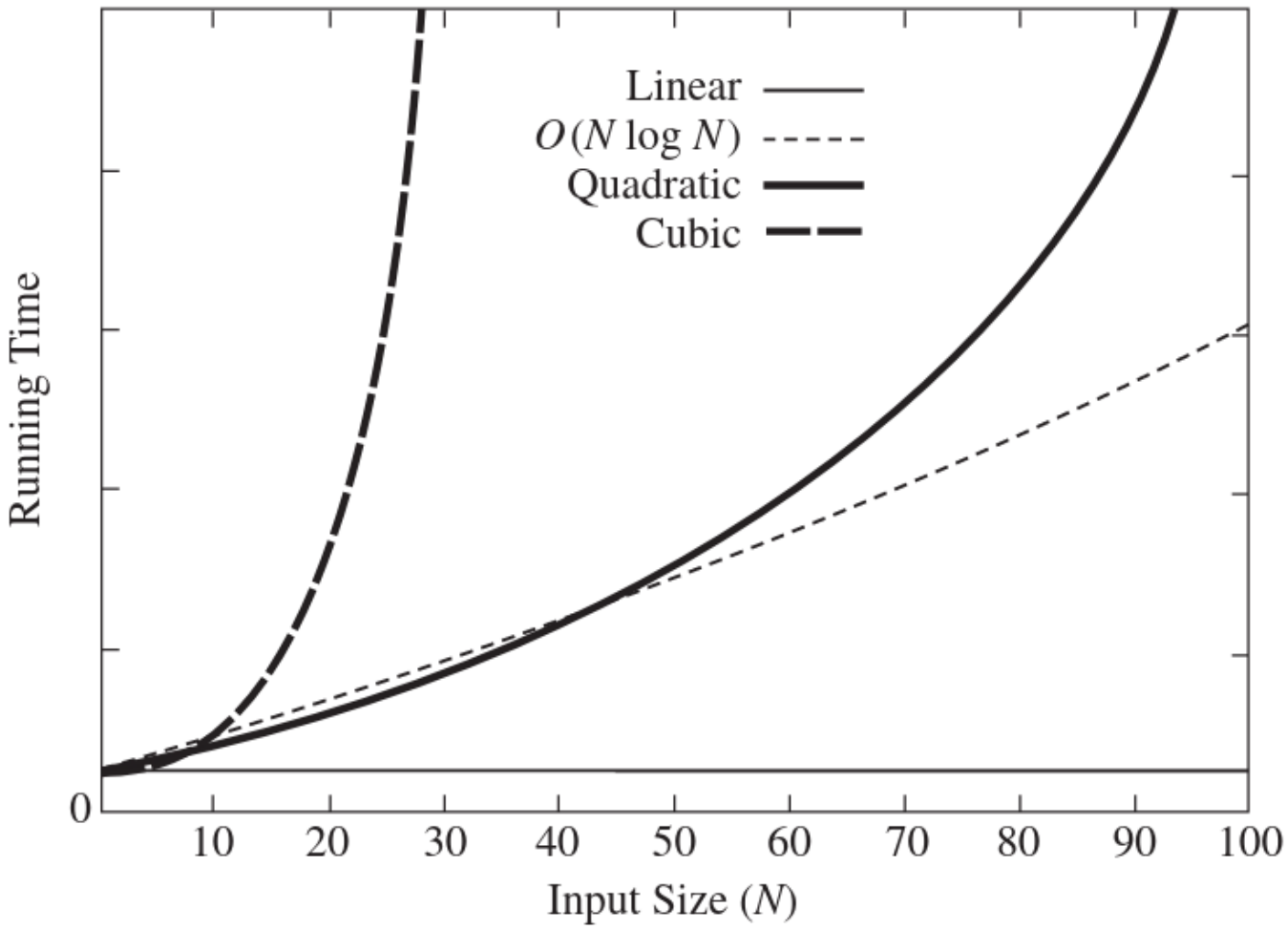
function	*name*	*typical value*	*approximation*
$\lfloor x \rfloor$	floor function	$\lfloor 3.14 \rfloor = 3$	x
$\lceil x \rceil$	ceiling function	$\lceil 3.14 \rceil = 4$	x
$\lg N$	binary logarithm	$\lg 1024 = 10$	$1.44 \ln N$
F_N	Fibonacci numbers	$F_{10} = 55$	$\phi^N / \sqrt{5}$
H_N	harmonic numbers	$H_{10} \approx 2.9$	$\ln N + \gamma$
$N!$	factorial function	$10! = 3628800$	$(N/e)^N$
$\lg(N!)$		$\lg(100!) \approx 520$	$N \lg N - 1.44N$

$$e = 2.71828\ldots$$
$$\gamma = 0.57721\ldots$$
$$\phi = (1 + \sqrt{5})/2 = 1.61803\ldots$$
$$\ln 2 = 0.693147\ldots$$
$$\lg e = 1/\ln 2 = 1.44269\ldots$$

GROWTH OF FUNCTIONS

Let us plot some of these functions and see how they grow with *N*.

Notice that some of them **intersect!**

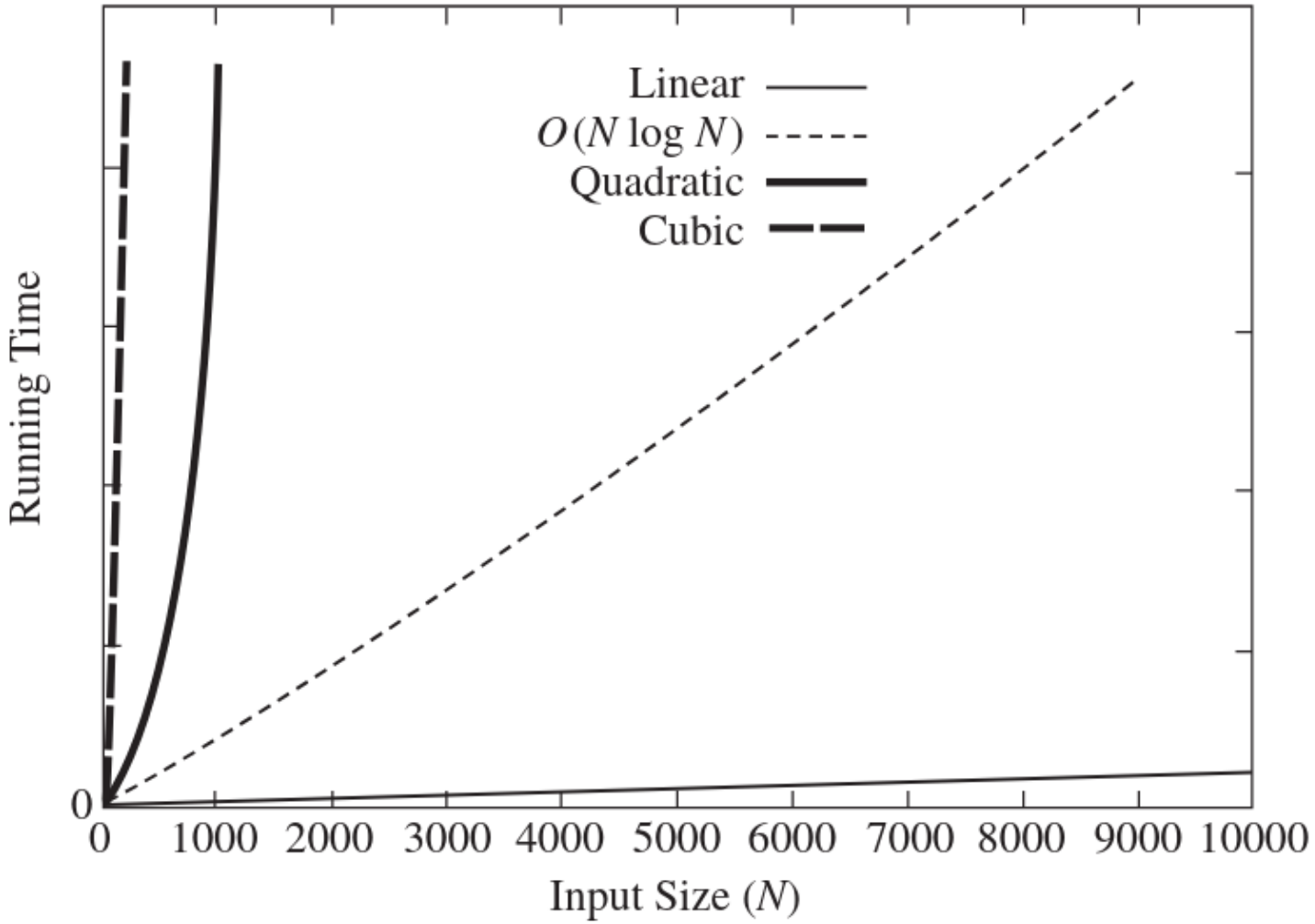


GROWTH OF FUNCTIONS

We are *particularly interested* in how they behave for **large values of N!**

But why?

This is also called the *asymptotic behavior, i.e.,* **when N goes to infinity.**



REFLECTION

Recall the following problem:

$$a + b + c = n$$

Find all sets of nonnegative integers (a, b, c) that sum to integer n $(n \geq 0)$

And the following two solutions, made by Alice and Bob (the pseudo code is presented, but both Bod and Alice use only pen and paper to solve the problem for a given n):

Bob's Solution

1. Try all combinations of (a, b, c)
2. If $a + b + c = n$, print(a, b, c)

Alice's Solution

1. For all (a, b) set $c = n - (a + b)$
2. If $c \geq 0$, print(a, b, c)

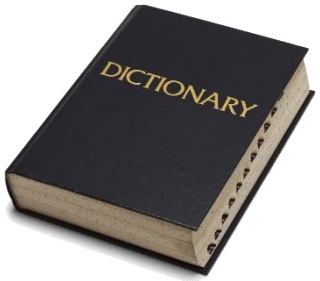
Discuss:

1. How many steps, given a value for n, will Bob take? What about Alice?

REFLECTION

Imagine that you have to find a word in a dictionary like the one on the right.

- How many steps does your algorithm take to find a given word, as a function of the number of pages?



https://www.collinsdictionary.com/dictionary/english/dictionary

AGENDA

Introduction to program analysis

Growth of functions

Notions of algorithm complexity

Benchmarking and efficiency

Data structures, Abstract Data Types (ADT), and Pre/Post Conditions

Linked Lists and Operations

Doubly Linked Lists and Operations

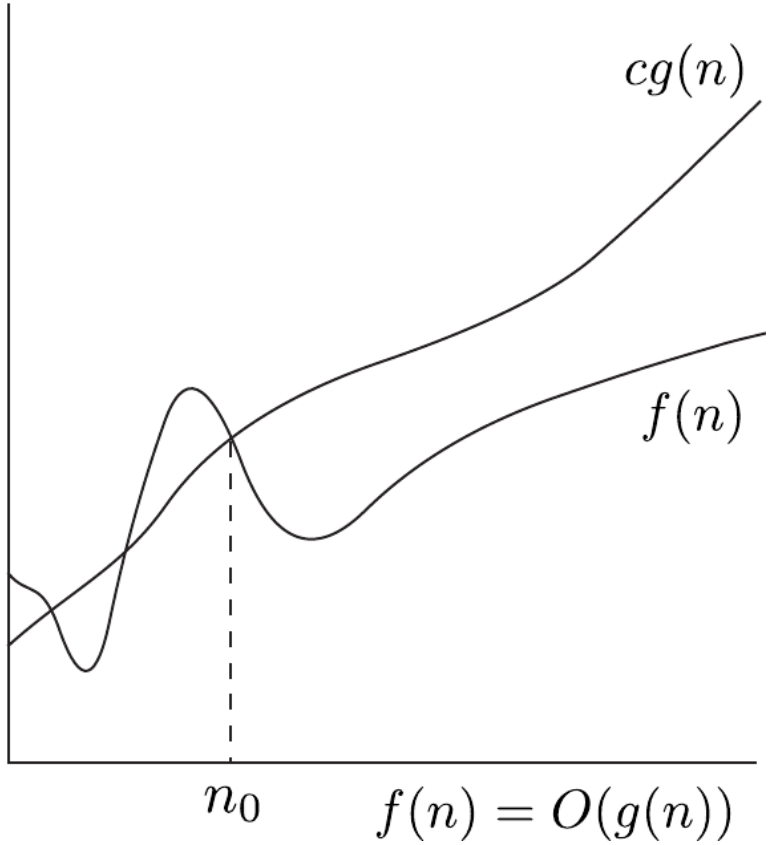
NOTIONS OF ALGORITHM COMPLEXITY

Our main tool will be ***O*-notation** (*read "Big-Oh"*) which formalizes the notion of **being proportional**, so we can capture the **overall performance** of an algorithm.

Definition:
A function $f(N)$ is said to be $O(g(N))$ if there exist constants c and n_0 such that

$$0 \leq f(N) \leq c * g(N)$$

for $c > 0$ and all $N \geq n_0$.



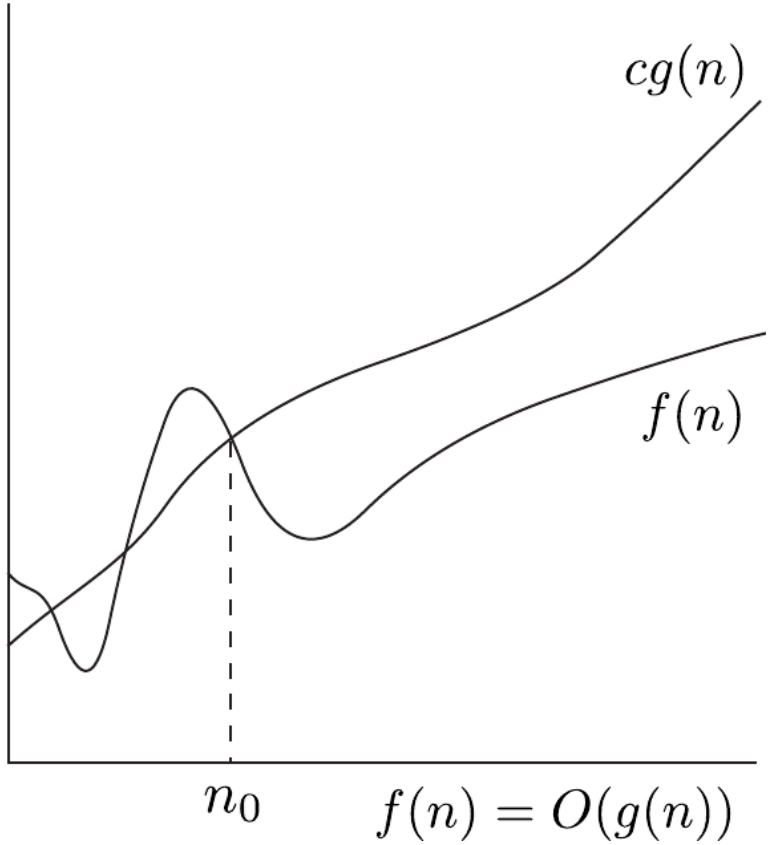
NOTIONS OF ALGORITHM COMPLEXITY

Another way to see the **same definition** if you are more inclined to **Calculus** and **continuous functions** can be found below.

Definition:
A function $f(N)$ is $O(g(N))$ if

$$\lim_{N\to\infty} \left(f(N)/g(N) \right) \in [0,\infty),$$

which limits the growth rate of f up to a constant times the growth rate of g.

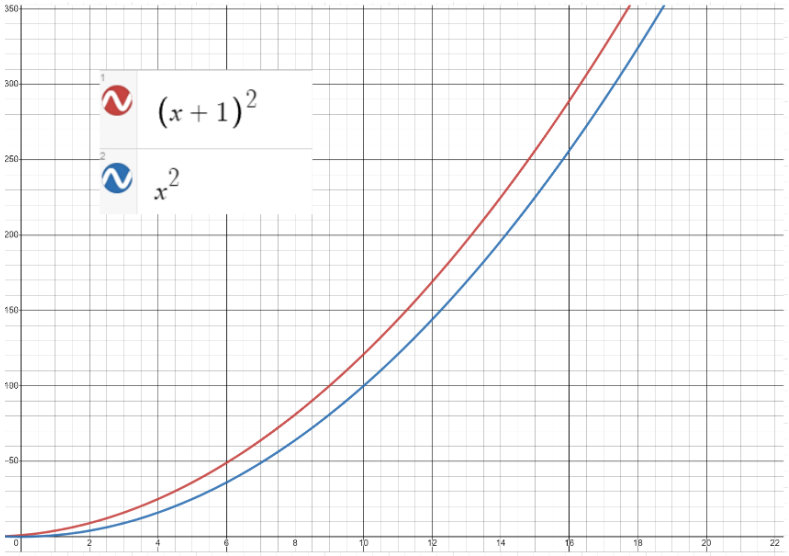


NOTIONS OF ALGORITHM COMPLEXITY

Example: Consider an algorithm that takes $(n+1)^2$ steps.

Why can we just claim that it is $O(n^2)$, and why not $O((n+1)^2)$? In other words, why do we care about simpler approximations and large n?

Consider the plot below and note that the blue function (our estimate for the algorithm's cost) is always below the red one. So it's not really a good estimate, right...?

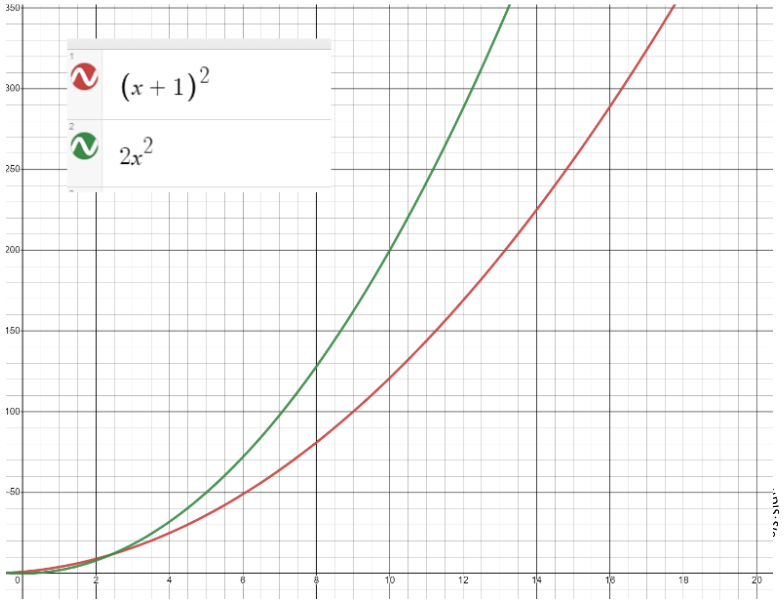


... but since we abstract away from the cost of each individual step taken by the algorithm, we make that assumption that it has some constant cost c.

So imagine that now each instruction takes $c = 2$ ns to run. Now the real cost of the algorithm is $2 * (n+1)^2$

Now we see in the plot below that the estimate is no longer that bad at all...

This is why it is often enough, in the worst case analysis, to consider the terms with the highest order in the expression describing the number of steps.



REFLECTION

Apply the definition of O(..) to the right and prove that $\mathrm{f(n)} = (n+1)^2$ is $O(n^2)$.

Hint: use the plots in the previous slide to find c and n_0.

Definition:
A function $f(N)$ is said to be $O(g(N))$ if there exist constants c and n_0 such that

$$0 \leq f(N) \leq c * g(N)$$

for $c > 0$ and all $N \geq n_0$.

NOTIONS OF ALGORITHM COMPLEXITY

We use the *O-notation* to simplify the analysis in a **technically precise sense:**

- To bound the error that we make when we **ignore** *constants and small terms.*
- To allow us to classify algorithms based on an **upper bound** of their running times.

We can now speak of an algorithm being **asymptotically** faster or slower than another.

NOTIONS OF ALGORITHM COMPLEXITY

Consider the following code fragment to compute $\Sigma\ i^3$ for $i = 1 .. N$.

```
1  int sum(int n) {
2      int i, partialSum = 0;
3
4      for (i = 1; i <= n; i++) {
5          partialSum = partialSum + (i * i * i);
6      }
7      return partialSum;
8  }
```

partialsum.c

- **Line 2** *costs* 1 initialization.
- **Line 4** *costs* 1 initialization, *(N+1)* comparisons and *N* increments.
- **Line 5** *costs 2* multiplications, 1 addition and 1 assignment per iteration.
- **Line 7** *costs* 1 assignment.
- **Total:** $T(N) = 6N+4$ operations.
- **Complexity:** $T(N) = O(N)$

NOTIONS OF ALGORITHM COMPLEXITY

Consider the following code fragment to compute $\Sigma\ i^3$ for $i = 1 .. N$.

```
1  int sum(int n) {
2      int i, partialSum = 0;
3
4      for (i = 1; i <= n; i++) {
5          partialSum = partialSum + (i * i * i);
6      }
7      return partialSum;
8  }
```

partialsum.c

- **Line 2** *costs O(1).*
- **Line 4** *costs O(N) in total.*
- **Line 5** *costs O(1)* per iteration.
- **Line 7** *costs O(1).*
- **Total:** *O(N) + 3O(1) = O(N).*
- **Complexity:** *T(N) = O(N)*

NOTIONS OF ALGORITHM COMPLEXITY

Some simple rules for **quick analysis:**

- *Loops:* multiply cost of the body by the number of iterations.
- *Nested loops:* the same, but be **careful** about numbers of iterations.
- *Conditionals:* cost of the test plus the larger of the two branches.

What about recursive functions?

We come back to these later.

REFLECTION (1/2)

Analyze the time complexity of **insertion sort**.

```c
/*
 * Insertion sort
 */

#include <assert.h>             /* assert */

void swap(int a, int b, int *x, int n) {
    /* pre-condition */
    /* a and b are within bounds of x array */
    assert(0 <= a && a < n && 0 <= b && b < n);

    /* post-condition */
    int t;  /* temporary variable for swapping */
    t = x[a];
    x[a] = x[b];
    x[b] = t;
}
```

insertion_sort.c

REFLECTION (2/2)

Analyze the time complexity of **insertion sort**.

```c
void insertion_sort(int n, int *x) {
    int i;  /* counter variable */
    int j;  /* counter variable */

    /* pre-condition */
    assert(0 <= n);

    /* post-condition: x[0..n] is sorted */
    for (i = 1; i < n; i = i + 1) {
        /* invariant: x[0..i-1] is sorted */
        j = i;
        /* insert next element j into
         * ordered position
         */
        for (; j > 0; j = j - 1) {
            if (x[j] > x[j - 1])
                break;          /* jth element is in correct position */
            else
                swap(j, j - 1, x, n);
        }
    }
}
```

insertion_sort.c

WHAT IS SPACE COMPLEXITY?

Space complexity of an algorithm is the total amount of memory it requires to execute, including:

1. **Input Storage:** Space needed to store input values.
2. **Auxiliary Space:** Extra space used during execution (e.g., temporary arrays, recursion stacks).
3. **Fixed Part:** Space required for constants and program instructions.

The answer is also given in big O of n where n is the primary parameter.

MENTIMETER – SPACE COMPLEXITY

What is the space complexity of the swap function?

```c
/*
 * Insertion sort
 */

#include <assert.h>             /* assert */

void swap(int a, int b, int *x, int n) {
    /* pre-condition */
    /* a and b are within bounds of x array */
    assert(0 <= a && a < n && 0 <= b && b < n);

    /* post-condition */
    int t;  /* temporary variable for swapping */
    t = x[a];
    x[a] = x[b];
    x[b] = t;
}
```

MENTIMETER – SPACE COMPLEXITY

What is the space complexity of the fibonacci function?

```
#include <iostream>
#include <vector>

int fibonacci(int n) {
    std::vector<int> fib(n);
    fib[0] = 0;
    fib[1] = 1;

    for (int i = 2; i < n; i++) {
        fib[i] = fib[i - 1] + fib[i - 2];  // Computing Fibonacci numbers iteratively
    }

    return fib[n - 1]; // Returning the last Fibonacci number
}

int main() {
    std::cout << fibonacci(10) << std::endl;
    return 0;
}
```

AGENDA

Introduction to program analysis

Growth of functions

Notions of algorithm complexity

Benchmarking and efficiency

Data structures, Abstract Data Types (ADT), and Pre/Post Conditions

BENCHMARKING AND EFFICIENCY

Running experiments in modern computers is **non-trivial**:

1. **Differences** in processor, memory, compiler, operating system
2. Processors **increase** performance depending on the **environment**
3. It may be hard to decide what a **realistic** workload is
4. It is **easy** (or even *tempting*) to benchmark on **optimistic conditions**

When running experiments, try to standardize **conditions** as much as possible. You can generate random inputs through functions srandom(time(0)) and random().

Important: useful for simple experiments, but **insecure** for everything else!

AGENDA

Introduction to program analysis

Growth of functions

Notions of algorithm complexity

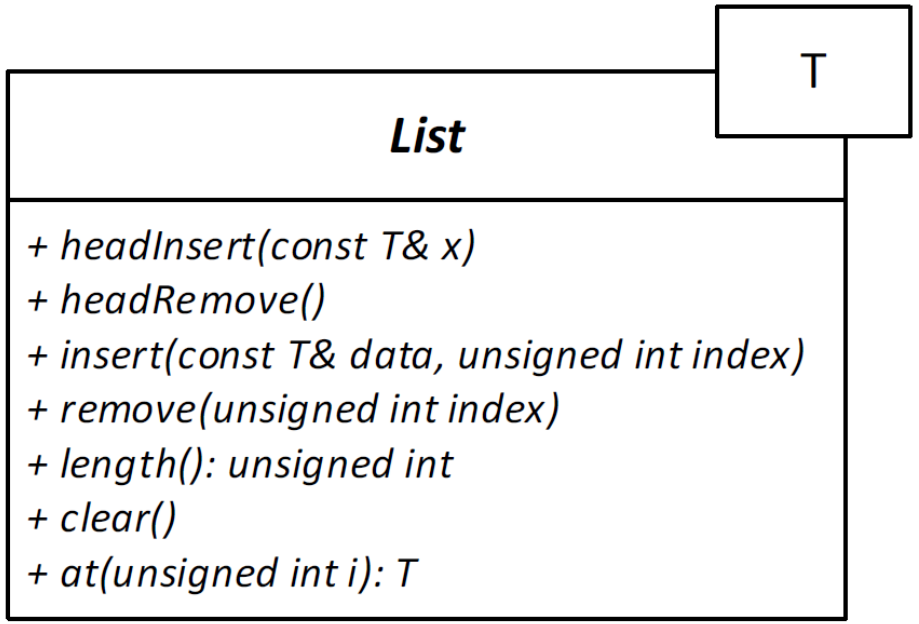
Benchmarking and efficiency

Data structures, Abstract Data Types (ADT), and Pre/Post Conditions

ABSTRACT DATA TYPES

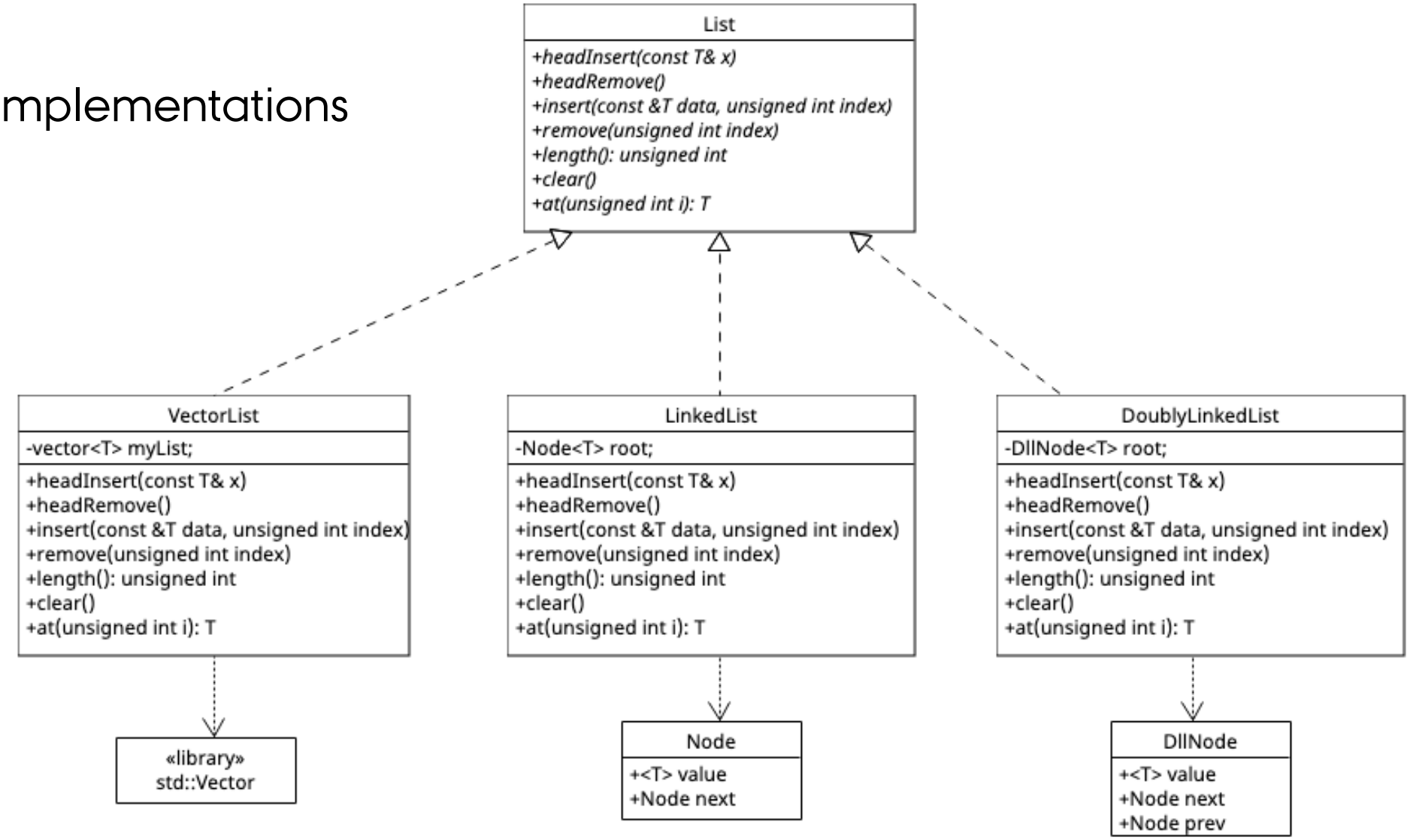
Definition:
An *abstract data type (ADT)* is a value or set of values that can be accessed only through an *interface*.

Example: List ADT



ABSTRACT DATA TYPES

Example: Implementations



DATA STRUCTURES

Storing and **accessing** data in **containers** is a basic step for the solution of problems. You know the **advantages** of an array:

- We can efficiently **store, access** and **change** data in an array
- What is the **time complexity** of these operations?

However, arrays have their **limitations** *(which?)*, and we need more **flexibility** when handling and **structuring** data.

Remark. In many problems, choosing the right **data structure** is critical for an elegant and efficient solution.

DATA STRUCTURES

Arrays have several **limitations**:

- Their capacity is in principle **fixed at declaration time**
- Elements cannot be easily **removed**, so it is hard to support dynamic data

What if the number of elements to store (or even the maximum) is **not known in advance**?

Notice that this situation happens frequently **in practice**, for example when you are reading from a file or keyboard input.

DATA STRUCTURES

You already know C-style implementations of data structures using **structs and pointers.** We will study a C++ implementation using higher-level abstraction through **classes and objects.**

We will define the abstract objects we want to manipulate and the operations we perform, initially without worrying about how to **represent** the data and to **implement** those operations.

Abstraction is the most fundamental tool to build more general and robust computer programs.

ABSTRACT DATA TYPES

Definition:
An *abstract data type (ADT)* is a value or set of values that can be accessed only through an *interface*.

You can also see ADTs as a **mathematical abstraction**, with public functions their **interface.**

The following slides show two alternative ways of defining abstract data types in C++

Regardless of the approach, the goal is the same: separate the interface from the implementation, and therefore hide (i.e., decouple) the implementation details from the client.

ABSTRACT DATA TYPES

Method 1: Use inheritance

point.h

```
1   #pragma once
2   class Point
3   {
4       /*Represents a point in a
5           two dimentional plane*/
6   public:
7       /*calculates the distance
8         from this point to a */
9       virtual float distance(Point& a) = 0;
10      virtual float getX() = 0;
11      virtual float getY() = 0;
12  };
13
14
```

test_point.cpp

```
vector<PointImpl> array = vector<PointImpl>();

for (int i=0; i<N; i++){
  array.push_back(PointImpl());
}

for (i = 0; i < N; i++) {
  for (j = i + 1; j < N; j++) {
    if (array[i].distance(array[j]) < d) {
      cnt++;
    }
  }
}
```

Pass by ref

pointimpl.h

```
1   #pragma once
2   #include "point.h"
3   #include <math.h>
4   class PointImpl:
5       public Point
6   {
7   private:
8       float x, y;
9   public:
10      PointImpl(float xp, float yp) {
11          x = xp;
12          y = yp;
13      }
14      PointImpl() {
15          x = 1.0 * rand() / RAND_MAX;
16          y = 1.0 * rand() / RAND_MAX;
17      }
18      ~PointImpl() {}
19      inline float getX() { return x; }
20      inline float getY() { return y; }
21      float distance(Point& a) {
22          float dx = x - a.getX();
23          float dy = y - a.getY();
24          return sqrt(dx * dx + dy * dy);
25      }
26  };
```

The distance only needs another Point, and not another PointImpl.

ABSTRACT DATA TYPES

Method 2: Use abstract classes (method preferred by the book).

```cpp
#ifndef _POINT_H_
#define _POINT_H_

class Point {
    private:
        float x, y;

    public:
        Point();
        ~Point();
        float distance(Point a);
};

#endif
```

point.h

point.cpp

point > point.cpp > ...

```cpp
#include <math.h>
#include "point.h"
#include <stdlib.h>

/* Constructor of a point. */
Point::Point() {
  x = 1.0 * rand() / RAND_MAX;
  y = 1.0 * rand() / RAND_MAX;
}

/* Destructor of a point. */
Point::~Point() {}

/* Compute the distance to another point. */
float Point::distance(Point a) {
  float dx = x - a.x;
  float dy = y - a.y;
  return sqrt(dx * dx + dy * dy);
}
```

point.cpp

ABSTRACT DATA TYPES

- The constructor generates a **random** point
- Destructor is **trivial** (memory allocation is automatic)
- The `distance()` method can access the **internal data.**
- We can also implement methods to **get** each field for a **cleaner Interface.**

```cpp
#include <math.h>
#include "point.h"

/* Constructor of a point. */
Point::Point() {
    x = 1.0*random()/RAND_MAX;
    y = 1.0*random()/RAND_MAX;
}

/* Destructor of a point. */
Point::~Point() {}

/* Compute the distance to another point. */
float Point::distance(Point a) {
    float dx = x - a.x;
    float dy = y - a.y;
    return sqrt(dx*dx + dy*dy);
}
```

point.cpp

ABSTRACT DATA TYPES

There are several new things in this testing code.

A *namespace* defines a *scope* for the symbols. We are *using* the default one.

We can mix with C library functions without a problem.

Dynamic memory allocation.

Calling a function or method.

C++ input/output

```
using namespace std;

#include <iostream>
#include <stdlib.h>
#include <time.h>
#include <assert.h>

#include "point.h"

int main(int argc, char *argv[]) {
    int i, j, cnt = 0;
    srandom(time(0));

    if (argc != 3) {
        cout << "Usage: test_point n d" << endl;
        return -1;
    }

    int N = atoi(argv[1]);
    float d = atof(argv[2]);

    Point *array = new Point[N];

    for (i = 0; i < N; i++) {
        for (j = i + 1; j < N; j++) {
            if (array[i].distance(array[j]) < d) {
                cnt++;
            }
        }
    }
    cout << cnt << " pairs within distance " << d << endl;
    delete [] array;
}
```

test_point.cpp

ABSTRACT DATA TYPES

Now let's improve the **interface** and **functionality** of the ADT:

- We changed `distance()` to be a *static* **class function,** no instance is needed to call it.
- We added *friend* operators for equality and printing.
- Notice the implementation of the new functions.

point_plus.h

```cpp
#ifndef _POINT_H_
#define _POINT_H_

class Point {
    private:
        float x, y;

    public:
        Point();
        ~Point();
        const float X() { return x; }
        const float Y() { return y; }
        static float distance(Point a, Point b);
        friend int operator==(Point a, Point b);
        friend ostream& operator<<(ostream& t, Point p);
};

#endif
```

```cpp
/* Compute the distance between points. */
float Point::distance(Point a, Point b) {
    float dx = a.X() - b.X();
    float dy = a.Y() - b.Y();
    return sqrt(dx*dx + dy*dy);
}

#define EPSILON 0.001f

int operator==(Point a, Point b) {
    return Point::distance(a, b) < EPSILON;
}

ostream& operator<<(ostream& t, Point p) {
    cout << "(" << p.x << " ," << p.y << ")";
    return t;
}
```

point_plus.cpp

ABSTRACT DATA TYPES

The updated test function shows how the **operators** are used:

- The **printing operator** can now be used as if `Point` was a basic type (line 25).
- The same applies to the **equality operator** (line 27).

We now can handle these objects without knowing how the class was **implemented**.

test_point_plus.cpp

```
1  using namespace std;
2
3  #include <iostream>
4  #include <stdlib.h>
5  #include <time.h>
6  #include <assert.h>
7
8  #include "point_plus.h"
9
10 int main(int argc, char *argv[]) {
11     int i, j, cnt = 0, equ = 0;
12
13     if (argc != 3) {
14         cout << "Usage: test_point_plus n d" << endl;
15         return -1;
16     }
17     srandom(time(0));
18
19     int N = atoi(argv[1]);
20     float d = atof(argv[2]);
21
22     Point *array = new Point[N];
23
24     for (i = 0; i < N; i++) {
25         cout << array[i] << endl;
26         for (j = i + 1; j < N; j++) {
27             if (array[i] == array[j]) {
28                 equ++;
29             }
30             if (Point::distance(array[i], array[j]) < d) {
31                 cnt++;
32             }
33         }
34     }
35     cout << cnt << " pairs within distance " << d << endl;
36     cout << equ << " pairs of equal points " << endl;
37     delete [] array;
38 }
39
```

EXAMPLE: INHERITANCE – APPROACH

- Specify **interface** using an **abstract base class**.
- Only **public pure virtual method** declarations.
- Still specify interface in separate header file.
- Separates interface and implementation, and allows multiple implementations of same interface **by inheritance.**

MaxHeap.h

```
C MaxHeap.h > ...
#pragma once
#include <vector>

using namespace std;

class MaxHeap {

public:
  // is the heap empty?
  virtual bool isEmpty() const = 0;

  // number of elements in the heap
  virtual int size() = 0;

  // add an element to the heap
  virtual void insert(const int x) = 0;

  // find the maximum element in the heap
  virtual const int findMax() const = 0;

  // delete and return the maximum element of the heap
  virtual int deleteMax() = 0;
};

```

EXAMPLE: INHERITANCE – APPROACH

- Implementation of specialized MaxHeap is now done using inheritance from the base class.
- Multiple implementations can co-exists – e.g. MaxHeapVector or MaxHeapList.
- Abstract base class cannot be instantiated but can be used as pointer and reference type.

MaxHeapVector.h

MaxHeap > C maxheapvector.h > ...

```cpp
#pragma once
using namespace std;
#include <vector>
#include "MaxHeap.h"

class MaxHeapVector : public MaxHeap
{
private:
    vector<int> v;

public:
    MaxHeapVector() {}
    virtual ~MaxHeapVector() {}

    virtual bool isEmpty() const {
        // TBD
    }

    virtual int size() {
        // TBD
    }

    virtual void insert(const int x) {
        // TBD
    }

};
```

Main.cpp

```cpp
#include "MaxHeap.h"
#include "MaxHeapVector.h"
#include <iostream>

int main(void){
    MaxHeap *m = new MaxHeapVector();
    m->insert(2);
    m->insert(3);
    m->insert(-1);
```

EXAMPLE: ABSTRACT CLASS – APPROACH

- One approach is to specify **interface** using standard **public** and **private** parts in a class.
- **Private** implementation.
- **Public** method declarations.
- Specify interface in separate header file.
- Separates interface and implementation, but does not allow multiple implementations of same interface.

MaxHeap.h

MaxHeap.h > ...

```cpp
#pragma once
#include <vector>

using namespace std;

class MaxHeap {

private:
  vector<int>* elements;

public:
  MaxHeap();

  // is the heap empty?
  bool isEmpty();

  // number of elements in the heap
  int size();

  // add an element to the heap
  void insert(const int x);

  // find the maximum element in the heap
  const int findMax();

  // delete and return the maximum element of the heap
  int deleteMax();
};
```

EXAMPLE: ABSTRACT CLASS – APPROACH

- Implementation using the private type (vector<int> in this case).
- Usage in the standard way – instantiating the defined class.
- Notice, no inheritance/ specialization – only single implementation.

MaxHeap.cpp

MaxHeap.cpp > ...

```cpp
#include "MaxHeap.h"

MaxHeap::MaxHeap() {
  elements = new vector<int>();
}

bool MaxHeap::isEmpty() {
  // TBD
}

int MaxHeap::size() {
  // TBD
}

void MaxHeap::insert(const int x) {
  // TBD
}
```

Main.cpp

ex_04.cpp > ...

```cpp
#include <iostream>
#include <stdlib.h>
#include "MaxHeap.h"
#include <cassert>

using namespace std;

int main(int argc, char* argv[]) {

  MaxHeap heap = MaxHeap();

  assert(heap.isEmpty());

  heap.insert(0);
  heap.insert(2);
```

PRE/POST CONDITIONS

Idea: use assertions to enforce contracts between a function/object and its clients.

- Discuss:
  - is this a good idea or a bad one?
  - What does a contract mean?
  - What are the alternatives?

Example:

- https://riptutorial.com/c/example/1810/precondition-and-postcondition

Pros:

- Actively enforce contract.
- Obeys fail-first principle.
- Serve as documentation: details what a function/object does without showing how.

Cons:

- More code = more maintenance.

AARHUS
UNIVERSITY